GO RUST PY Live-Coding Cheatsheet

Same task, three languages, side by side · syntax, containers, sort, strings, producer → consumers · one A3 sheet

syntax key: `keyword` `i64 Type` `"str"` `builtin` `// comment`
★ use constantly · E0xxx Rust compiler error · rows aligned by topic

01 Skeleton · Fast I/O

Go

```go
package main
import ("bufio"; "fmt"; "os"; "strconv"; "strings")
func main() {
    in := bufio.NewReader(os.Stdin)
    out := bufio.NewWriter(os.Stdout)
    defer out.Flush()                    // or nothing prints! ★
    var n int
    fmt.Fscan(in, &n)                    // tokens
    a := make([]int, n)
    for i := range a { fmt.Fscan(in, &a[i]) }
    fmt.Fprintln(out, n, a[0])
}
sc := bufio.NewScanner(os.Stdin)         // line based
for sc.Scan() { f := strings.Fields(sc.Text()) }  // + strconv.Atoi
```

Rust

```rust
use std::io::{self, Read, Write, BufRead};
fn main() {
    let mut s = String::new();
    io::stdin().read_to_string(&mut s).unwrap();
    let mut it = s.split_ascii_whitespace()
        .map(|x| x.parse::<i64>().unwrap());
    let n = it.next().unwrap() as usize;
    let a: Vec<i64> = it.by_ref().take(n).collect();
    let out = io::stdout();
    let mut w = io::BufWriter::new(out.lock());  // ★
    writeln!(w, "{} {}", n, a[0]).unwrap();
}
for line in io::stdin().lock().lines() {     // line based
    let f: Vec<i64> = line.unwrap().split_whitespace()
        .map(|x| x.parse().unwrap()).collect(); }
```

Python

```python
import sys
input = sys.stdin.readline               # fast input ★
n = int(input())
a = list(map(int, input().split()))
x, y = map(int, input().split())
print(n, a[0])
# all at once
data = sys.stdin.read().split(); it = iter(data)
n = int(next(it)); a = [int(next(it)) for _ in range(n)]
out = []; out.append(f"{n} {a[0]}")      # fast output
sys.stdout.write("\n".join(out) + "\n")
sys.setrecursionlimit(1 << 25)           # deep DFS
```

02 Variables · Types · Casting

SYNTAX

Go

```go
var x int = 5;  y := 10;  const Mod = 1_000_000_007
var a, b int;  a, b = b, a                  // swap
x := int64(y) * int64(z)                    // explicit widen
f := float64(n) / float64(m);  i := int(f)  // trunc
n, err := strconv.Atoi(s);  s = strconv.Itoa(n)
d := int(c - '0');  c := byte('a' + i)
math.MaxInt64  math.MinInt  math.Inf(1)
(x%m + m) % m                               // non-neg mod
min(a, b)  max(a, b)  // int is 64-bit: no overflow worry
```

Rust

```rust
let x = 5;  let mut y: u64 = 10;  const MOD: i64 = 1_000_000_007;
let (a, b) = (b, a);                        // swap (shadow)
let i = y as i32;  let idx = n as usize; // index = usize ★
let f = a as f64 / b as f64;  let i = f as i64;  // trunc
let n: i32 = s.parse().unwrap();  n.to_string()
let d = (c as u8 - b'0') as usize;  (b'a' + i as u8) as char
i64::MAX  i64::MIN  f64::INFINITY  x.rem_euclid(m)
x.abs_diff(y)  x.checked_add(y)  x.min(y)  x.pow(3)
```

Python

```python
x = 5;  y = 10;  MOD = 10**9 + 7          # ints are unbounded ★
a, b = b, a                               # swap
f = a / b;  q = a // b;  r = a % b     # // floors; % non-neg (m>0)
int(f)  round(f)  float(n)  int("42")  str(n)
ord(c) - ord('0')   chr(ord('a') + i)
float('inf')  -float('inf')  math.inf
divmod(a, b)  abs(x)  pow(b, e, m)  x ** 0.5
isinstance(x, int)  type(x)
```

03 Control Flow

SYNTAX

Go

```go
if x > 0 { } else if x < 0 { } else { }
if v, ok := m[k]; ok { }                    // init stmt
for i := 0; i < n; i++ { }    for i := range n { }
for i, v := range a { }     for k, v := range m { }
for cond { }  /* while */     for { break }  /* loop */
outer: for .. { for .. { continue outer } }
switch x { case 1, 2: ..; case 3: ..; default: .. }
switch { case x < 0: ..; case x < 10: .. }  // if-chain
```

Rust

```rust
if x > 0 { } else if x < 0 { } else { }
let s = if x > 0 { "p" } else { "n" };   // expression
for i in 0..n { }    for i in (0..n).rev() { }
for (i, v) in a.iter().enumerate() { }    for v in &a { }
while cond { }    loop { break; }    while let Some(x) = st.pop() { }
'outer: for .. { for .. { continue 'outer; } }
match x { 0 => a, 1 | 2 => b, 3..=9 => c, n if n < 0 => d, _ => e }
if let Some(v) = m.get(&k) { }    let Some(v) = o else { return };
```

Python

```python
if x > 0: ...  elif x < 0: ...  else: ...
s = "p" if x > 0 else "n"                 # ternary
for i in range(n): ...    for i in range(n - 1, -1, -1): ...
for i, v in enumerate(a): ...    for k, v in d.items(): ...
while cond: ...    for ...: ... else: ... # else runs if no break
match x:                                  # 3.10+
    case 0 | 1: ...    case [a, b, *rest]: ...    case _: ...
```

04 Functions · Closures

SYNTAX

Go

```go
func add(a, b int) int { return a + b }
func divmod(a, b int) (int, int) { return a / b, a % b }
func sum(xs ...int) int { }    sum(v...)        // variadic
sq := func(x int) int { return x * x }
cnt := 0;  inc := func() { cnt++ }            // captures by ref
var dfs func(u int)                       // recursive closure ★
dfs = func(u int) { .. dfs(v) .. }
defer f.Close()    defer func(){ recover() }()
func Max[T cmp.Ordered](a, b T) T { .. }      // generics
```

Rust

```rust
fn add(a: i32, b: i32) -> i32 { a + b }       // last expr returns
fn sum(v: &[i64]) -> i64 { v.iter().sum() }   // take slices ★
fn f(v: &mut Vec<i32>) { v.push(1); }         // mutate
let sq = |x: i32| x * x;    let mut inc = || cnt += 1;
let f = move |x| x + off;                     // take ownership
fn dfs(u: usize, g: &[Vec<usize>], vis: &mut [bool]) { }
fn big<T: Ord + Copy>(a: T, b: T) -> T { if a > b {a} else {b} }
fn apply(f: impl Fn(i32) -> i32, x: i32) -> i32 { f(x) }
```

Python

```python
def add(a, b): return a + b
def divmod_(a, b): return a // b, a % b       # tuple return
q, r = divmod_(7, 2)
def f(*args, **kw): ...    f(*lst)            # varargs / unpack
sq = lambda x: x * x
def dfs(u):                                   # recursion
    for v in g[u]: dfs(v)
def inc(): nonlocal cnt; cnt += 1             # closure write
from functools import lru_cache
@lru_cache(None)                              # memoize ★
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
```

05 Array · List · Vec · Slice

COLLECTIONS

Go

```go
var a [26]int              // array: fixed, value type
s := []int{1, 2, 3};  s := make([]int, n)   /* n zeros */
s := make([]int, 0, n)     // len 0 cap n ★
g := make([][]int, n); for i := range g { g[i] = make([]int, m) }
s = append(s, x)           // ALWAYS reassign ★
len(s)  s[i]  s[len(s)-1]   s = s[:len(s)-1]  /* pop */
s = append(s[:i], s[i+1:]...)              // remove i
s[i], s[j] = s[j], s[i]     slices.Reverse(s)
slices.Contains(s, x)  slices.Index(s, x)  slices.Max(s)
c := slices.Clone(s)       // copy; `=` shares memory ★
```

Rust

```rust
let arr = [0u8; 26];       // array: fixed, Copy
let mut v = vec![1, 2, 3];  let v = vec![0; n];
let v = Vec::with_capacity(n);
let g = vec![vec![0; m]; n];                // 2D ★
v.push(x)  v.pop()/*Option*/  v.len()  v[i]  v.last()
v.insert(i, x)  v.remove(i)  v.swap_remove(i)
v.swap(i, j)  v.reverse()  v.contains(&x)
v.iter().position(|&e| e == x)  v.iter().max()
let c = v.clone();          // deep copy; `=` moves ★
v.retain(|&x| x > 0)  v.dedup()  v.truncate(k)
```

Python

```python
a = [1, 2, 3];  a = [0] * n;  g = [[0] * m for _ in range(n)]  # 2D
a = [0] * n;  g = [[0]*m]*n   # BUG: rows alias ★
a.append(x)  a.pop()  a.pop(0)  /*O(n)*/  a.insert(i, x)
len(a)  a[i]  a[-1]  a.index(x)  a.count(x)  x in a
a.remove(x)  del a[i]  a.reverse()  a[::-1]
a[i], a[j] = a[j], a[i]    max(a)  min(a)  sum(a)
b = a[:]  b = a.copy()  b = list(a)   # copy; `=` aliases ★
import copy; copy.deepcopy(g)          # nested
a.extend(b)  a += b  a * 3  a.clear()
```

06 Slicing · Concat · Conversion

COLLECTIONS

Go

```go
s[1:3]  s[:k]  s[k:]        // shares backing array ★
c := append([]int{}, a...); c = append(c, b...)  // safe
c := slices.Concat(a, b)    // 1.22+ new slice ★
a = append(a, b...)         // may alias if cap fits
flat := slices.Concat(grid...)
s := arr[:]                 // array -> slice
arr := [3]int(s)            // slice -> array (len match)
copy(dst, src)              // min(len) elems
strs := strings.Split(s, " ")   strings.Join(strs, ",")
[]byte(s)  string(bs)  []rune(s)  string(rs)
```

Rust

```rust
&v[1..3]  &v[..k]  &v[k..]  v.split_at(k)
let c = [&a[..], &b[..]].concat();        // keep both ★
let c = [a, b].concat();                  // moves a, b
a.extend_from_slice(&b) /* keep b */  a.append(&mut b) /* drain b */
let c: Vec<_> = a.iter().chain(&b).cloned().collect();
let flat: Vec<i32> = grid.concat();
let v = arr.to_vec();   let arr: [i32; 3] = v.try_into().unwrap();
v.windows(2)  v.chunks(3)  v.iter().zip(&w)
let s: String = cs.iter().collect();   s.chars().collect::<Vec<_>>()
s.as_bytes()  String::from_utf8(bytes)
```

Python

```python
a[1:3]  a[:k]  a[k:]  a[::2]  a[::-1]    # copies ★
c = a + b                                 # new list
a += b  a.extend(b)                       # in place
flat = [x for row in g for x in row]
flat = list(itertools.chain.from_iterable(g))
list(zip(a, b))   list(zip(*g))  # transpose ★
list("abc")  "".join(chars)  s.split()  ",".join(map(str, a))
list(range(n))  tuple(a)  set(a)  dict(pairs)
s.encode()  b.decode()  bytes([65])  bytearray(b"ab")
```

07 Map · Set

COLLECTIONS

Go

```go
m := map[string]int{};  m := make(map[string]int, n)
m[k] = v;  v := m[k]  /* zero if missing */   delete(m, k)
v, ok := m[k];  if _, ok := m[k]; !ok { }
m[k]++                      // counting ★
m[k] = append(m[k], x)      // adjacency ★
for k, v := range m { }     // random order!
keys := slices.Sorted(maps.Keys(m))      // 1.23
seen := map[int]bool{}; seen[x] = true; if seen[x] { }
m := map[[2]int]int{}       // struct / array keys ok, slice NO
var m map[K]V  // nil: write PANICS -> use make ★
```

Rust

```rust
use std::collections::{HashMap, HashSet, BTreeMap};
let mut m: HashMap<String, i32> = HashMap::new();
m.insert(k, v);  m.get(&k)/*Option<&V>*/  m[&k]/*panics*/
m.contains_key(&k)  m.remove(&k)  m.len()
*m.entry(k).or_insert(0) += 1;            // counting ★
m.entry(k).or_default().push(x);          // adjacency ★
for (k, v) in &m { }   m.keys()  m.values()
let mut s = HashSet::new(); s.insert(x); s.contains(&x)
BTreeMap: sorted; b.range(k..).next()  b.first_key_value()
f64 can't be key; tuples / Vec / String can
```

Python

```python
d = {};  d = {"a": 1};  d = dict();  d[k] = v;  d[k]  # KeyError
d.get(k, 0)   k in d   del d[k]   d.pop(k, None)   len(d)
d[k] = d.get(k, 0) + 1                    # counting
from collections import defaultdict, Counter
dd = defaultdict(int); dd[k] += 1         # ★
dd = defaultdict(list); dd[k].append(x)   # adjacency ★
cnt = Counter(a); cnt.most_common(3)
for k, v in d.items():   d.keys()  d.values()  sorted(d)
s = set(); s.add(x); s.discard(x); x in s; a | b; a & b; a - b
keys: tuples ok, lists NOT (unhashable); dict keeps insert order
```

08 Sorting

COLLECTIONS

Go

```go
sort.Ints(a)  sort.Strings(s)  slices.Sort(a)
sort.Sort(sort.Reverse(sort.IntSlice(a)))      // desc
sort.Slice(ps, func(i, j int) bool { return ps[i].s < ps[j].s })
sort.SliceStable(ps, func(i, j int) bool {     // multi-key ★
    if ps[i].s != ps[j].s { return ps[i].s > ps[j].s }
    return ps[i].n < ps[j].n })
slices.SortFunc(ps, func(a, b P) int {
    return cmp.Or(cmp.Compare(b.s, a.s), cmp.Compare(a.n, b.n)) })
i := sort.SearchInts(a, x)                     // lower_bound
sort.Search(n, func(i int) bool { return a[i] >= x })
```

Rust

```rust
v.sort();  v.sort_unstable();  v.sort_by(|a, b| b.cmp(a));  // desc
v.sort_by_key(|p| p.1);   v.sort_by_key(|&x| Reverse(x));
v.sort_by(|a, b| b.1.cmp(&a.1).then(a.0.cmp(&b.0)));   // multi ★
v.sort_by_key(|p| (Reverse(p.1), p.0.clone()));
f.sort_by(|a, b| a.total_cmp(b));          // f64
idx.sort_by_key(|&i| a[i]);                // argsort
v.binary_search(&x)  v.partition_point(|&x| x < t)  // lower_bound
v.iter().max_by_key(|p| p.1)   v.select_nth_unstable(k)
```

Python

```python
a.sort()  sorted(a)  a.sort(reverse=True)
a.sort(key=lambda p: p[1])
a.sort(key=lambda p: (-p[1], p[0]))        # multi-key ★ desc, asc
sorted(d.items(), key=lambda kv: kv[1])
from operator import itemgetter, attrgetter; key=itemgetter(1)
sorted(words, key=len)   max(a, key=f)   min(a, key=f)
import bisect; bisect.bisect_left(a, x)      # lower_bound
bisect.bisect_right(a, x)  bisect.insort(a, x)
sorted(s, key=str.lower)   stable sort by default
```

09 Strings · Builder · Bytes

COLLECTIONS

Go

```go
len(s) /*bytes*/   s[i] /*byte*/   []rune(s)[i] /*char*/
for i, r := range s { }     // runes
strings.Contains  HasPrefix  Index  ToUpper  TrimSpace
strings.Split(s, ",")  Fields(s)  Join(xs, " ")  Repeat
strings.Replace(s, a, b, -1)  strings.Cut(s, "=")
s + t  /* ok once */       // in loops use Builder ★
var sb strings.Builder; sb.WriteString("x"); sb.WriteByte('y')
sb.String()   fmt.Fprintf(&sb, "%d", n)
var buf bytes.Buffer; buf.WriteString("x"); buf.Bytes()
unicode.IsDigit(r)  'a' <= c && c <= 'z'  c - '0'
```

Rust

```rust
s.len() /*bytes*/  s.chars().count()  s.as_bytes()[i]  // ★
let cs: Vec<char> = s.chars().collect(); cs[i]
s.contains("x") .starts_with .find .to_uppercase .trim()
s.split(',')  .split_whitespace()  .lines()  .split_once('=')
s.replace("a", "b")  .repeat(3)  .chars().rev().collect::<String>()
let mut t = String::new(); t.push('x'); t.push_str("yz");   // ★
t += &u;   let w = format!("{t}{u}");   // keep both
let mut b: Vec<u8> = Vec::new(); b.extend_from_slice(s.as_bytes());
b.push(b'!');  String::from_utf8(b).unwrap()
s.parse::<i32>()  n.to_string()  c.is_ascii_digit()  c.to_digit(10)
```

Python

```python
len(s)  s[i]  s[-1]  s[1:3]  s[::-1]      # str immutable
s.upper() .lower() .strip() .startswith() .find(x) /* -1 */
s.split(",")  s.split()  " ".join(xs)  s.replace(a, b)
s.isdigit() .isalpha() .isalnum()     c in "aeiou"
parts = []; parts.append(x); "".join(parts)   # builder ★
from io import StringIO; buf = StringIO(); buf.write("x")
ba = bytearray(); ba += b"ab"; ba.append(65); bytes(ba)
b = s.encode(); b[0]  /* int */    b.decode()
f"{x:>5} {y:.2f} {n:05d} {v!r}"           # format ★
s.zfill(5)  s.count("a")  s.ljust(5)  ord(c)  chr(n)
```

10 Struct · Methods · Interface / Trait / Class

TYPES

Go

```go
type P struct{ X, Y int }
p := P{1, 2}; pp := &P{X: 1}; p2 := p  // copy!
func (p P) Dist() int { return p.X + p.Y }
func (p *P) Shift(d int) { p.X += d }   // mutates ★
type Shape interface { Area() float64 }
func (c Circle) Area() float64 { .. }   // implicit impl
type Color int; const (Red Color = iota; Green)
type Named struct{ P; Tag string }      // embedding
```

Rust

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
struct P { x: i32, y: i32 }
impl P { fn new(x: i32, y: i32) -> Self { Self { x, y } }
    fn dist(&self) -> i32 { self.x + self.y }
    fn shift(&mut self, d: i32) { self.x += d; } }
enum Shape { Circle(f64), Rect { w: f64, h: f64 } }
trait Area { fn area(&self) -> f64; }
impl Area for P { fn area(&self) -> f64 { 0.0 } }
fn show(a: &impl Area) { }   fn show(a: &dyn Area) { }
impl Ord for St { fn cmp(&self, o: &Self) -> Ordering { .. } }
```

Python

```python
class P:
    def __init__(self, x, y): self.x, self.y = x, y
    def dist(self): return self.x + self.y
    def __repr__(self): return f"P({self.x},{self.y})"
    def __lt__(self, o): return self.x < o.x     # for sort/heap
from dataclasses import dataclass
@dataclass(order=True)                          # ★
class P: x: int; y: int = 0
Pt = collections.namedtuple("Pt", "x y"); p = Pt(1, 2); p.x
class Dog(Animal): def speak(self): return "woof"  # inherit
```

11 Errors · Option · Null

TYPES

Go

```go
n, err := strconv.Atoi(s)
if err != nil { return 0, err }         // the idiom ★
return 0, fmt.Errorf("ctx: %w", err)    // wrap
errors.Is(err, ErrX)  errors.As(err, &e)
var p *P   // nil pointer; nil slice/map ok to read
v, ok := m[k]   x, ok := i.(T)   v, ok := <-ch
panic("x")  defer func(){ recover() }()
```

Rust

```rust
Some(x) / None    Ok(x) / Err(e)
o.unwrap() .expect("m") .unwrap_or(0) .unwrap_or_default()
o.map(|x| ..) .and_then(f) .ok_or(e) .is_some()
r.ok() .map_err(f) .unwrap_or(d)   r? o?   // early return ★
let Some(v) = o else { return; };
fn f() -> Result<T, Box<dyn Error>> { let n: i32 = s.parse()?; .. }
m.get(&k).copied().unwrap_or(0)
```

Python

```python
try: n = int(s)
except ValueError as e: ...
except (A, B): ...
else: ...  finally: ...
raise ValueError("bad")
x = d.get(k)   if x is None: ...    x or default
assert cond, "msg"
try: a[i] except IndexError: ...
```

12 Producer → Multiple Consumers → Sort Results

CONCURRENCY

Go

```go
type R struct{ id, val int }
jobs := make(chan int, 100)
results := make(chan R, 100)
var wg sync.WaitGroup
for w := 0; w < 4; w++ {                  // N consumers
    wg.Add(1)
    go func() { defer wg.Done()           // 1 job -> 1 worker
        for j := range jobs { results <- R{j, j * j} }
    }()
}
go func() {                               // 1 producer
    for i := 0; i < 20; i++ { jobs <- i }
    close(jobs)                           // no more work ★
}()
go func() { wg.Wait(); close(results) }() // after ALL workers ★
var all []R
for r := range results { all = append(all, r) }
sort.Slice(all, func(i, j int) bool {     // final sort ★
    return all[i].id < all[j].id })
for _, r := range all { fmt.Println(r.id, r.val) }
```

Rust

```rust
use std::sync::{mpsc, Arc, Mutex};  use std::thread;
let (tx, rx) = mpsc::channel::<i32>();        // jobs
let rx = Arc::new(Mutex::new(rx));            // 1 rx, shared ★
let (rtx, rrx) = mpsc::channel();             // results
let hs: Vec<_> = (0..4).map(|id| {            // N consumers
    let (rx, rtx) = (Arc::clone(&rx), rtx.clone());
    thread::spawn(move || loop {
        let job = rx.lock().unwrap().recv(); // lock ends at ;
        match job { Ok(j) => rtx.send((j, j * j)).unwrap(),
                    Err(_) => break }         // closed -> exit
    })
}).collect();
drop(rtx);                                    // workers hold rest
for i in 0..20 { tx.send(i).unwrap(); }       // 1 producer
drop(tx);                                     // close -> stop ★
let mut all: Vec<(i32, i32)> = rrx.iter().collect();
for h in hs { h.join().unwrap(); }
all.sort_by_key(|&(id, _)| id);               // final sort ★
for (id, v) in &all { println!("{id} {v}"); }
```

Python

```python
import threading, queue
from concurrent.futures import ThreadPoolExecutor
q = queue.Queue()                             # thread-safe ★
results = []; lock = threading.Lock()
def worker():
    while True:
        j = q.get()                           # blocks
        if j is None: break                   # sentinel = stop ★
        with lock: results.append((j, j * j))
        q.task_done()
ts = [threading.Thread(target=worker) for _ in range(4)]
for t in ts: t.start()
for i in range(20): q.put(i)                  # 1 producer
for _ in ts: q.put(None)                      # one sentinel per worker
for t in ts: t.join()
results.sort(key=lambda r: r[0])              # final sort ★
for id_, v in results: print(id_, v)
with ThreadPoolExecutor(4) as ex:             # simpler; keeps order
    results = list(ex.map(lambda j: (j, j * j), range(20)))
# CPU-bound: multiprocessing.Pool(4).map(f, jobs)  (GIL)
```

13 Heap · Deque · Queue

COLLECTIONS

Go

```go
type IntHeap []int   // Len Less Swap + Push/Pop(ptr) ★
func (h IntHeap) Less(i, j int) bool { return h[i] < h[j] }
func (h *IntHeap) Push(x any) { *h = append(*h, x.(int)) }
func (h *IntHeap) Pop() any { o := *h; n := len(o)
    x := o[n-1]; *h = o[:n-1]; return x }
h := &IntHeap{}; heap.Init(h); heap.Push(h, 3)
x = heap.Pop(h).(int);  (*h)[0] /* peek */
q := []int{}; q = append(q, x); x, q = q[0], q[1:]  // queue
st = append(st, x); x, st = st[len(st)-1], st[:len(st)-1]
```

Rust

```rust
use std::collections::{BinaryHeap, VecDeque}; use std::cmp::Reverse;
let mut h = BinaryHeap::new();    // MAX-heap
h.push(x)  h.pop()  h.peek()  h.len()
h.push(Reverse(x)); if let Some(Reverse(x)) = h.pop() { }  // min
h.push(Reverse((dist, node)));    // Dijkstra ★
let mut q = VecDeque::new();      // BFS
q.push_back(x)  q.pop_front()  q.push_front(x)  q.front()
let mut st = vec![]; st.push(x); st.pop();   // stack
```

Python

```python
import heapq
h = []; heapq.heappush(h, x); x = heapq.heappop(h); h[0]  # min-heap
heapq.heappush(h, -x)                         # max-heap trick ★
heapq.heappush(h, (dist, node))               # Dijkstra
heapq.heapify(a)  heapq.nlargest(k, a)
from collections import deque
q = deque(); q.append(x); q.popleft(); q.appendleft(x); q[0]
st = []; st.append(x); st.pop(); st[-1]       # stack
```

14 Traps · Must Remember

TRAPS

- unused `var`/`import` = compile error → `_ = x`
- `append` may not update caller's slice → return it
- `map` iteration order is random
- closing a channel twice / sending after close = panic
- `nil map` write panics; s[a:b] shares memory
- `int` is 64-bit; x++ is a statement

- `E0502` borrow conflict → clone / copy value first
- `E0382` use after `move` → iterate &v, .iter(), .clone()
- `usize - usize` underflows (panic in debug) → `i64` / `abs_diff`
- indexes must be `usize;` `as` casts everywhere
- `String[i]` not allowed → `as_bytes()[i]` or `chars()`
- sort needs `Ord`: `f64` → `total_cmp` / `partial_cmp`

- `[[0]*m]*n` aliases rows → `list comp`
- default recursion limit `1000` → `setrecursionlimit`
- `input()` slow → `sys.stdin.readline`
- `a = b` aliases lists/dicts → `copy()`
- `is` vs `==`; mutable default arg `def f(x=[])`
- integer division `//`, true division `/`; strings immutable